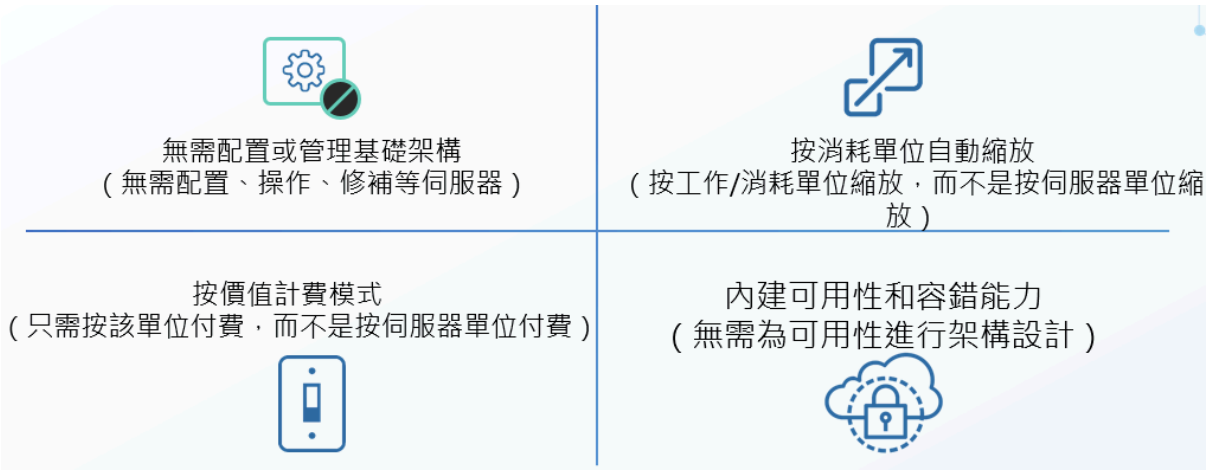
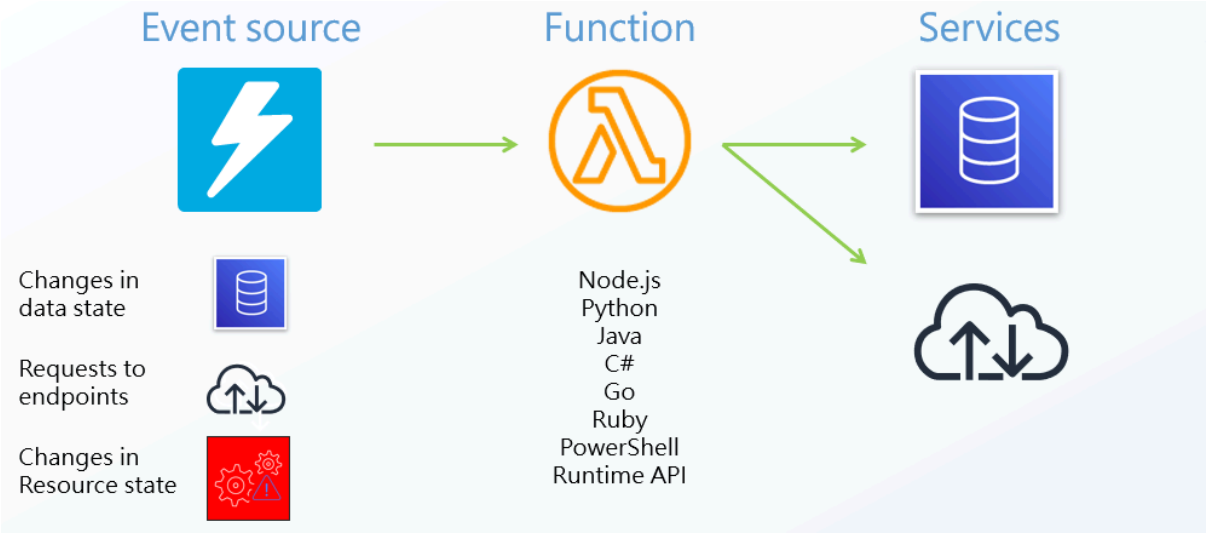
課程5 - AWS Lambda + EventBridge

1-1：Serverless 的定義

1-1-1：說明：當我們說無伺服器時，我們的意思是消除伺服器操作這種無差別的繁重工作。對於客戶來說，這是一個重要的區別，因為它允許客戶專注於應用程式的構建，而不是支援應用程式的基礎設施的管理和擴展：



1-1-2：Lambda在應用架構上的組件：



Event Source（觸發來源）	數據狀態變更（Changes in data state）、端點請求（Requests to endpoints）、資源狀態變更（Changes in Resource state）
Function（函數）	支援 Node.js、Python、Java、C#、Go、Ruby、PowerShell，以及 Runtime API 自定義執行環境
Services（後端服務）	Lambda 函數可調用 AWS 各種服務完成業務邏輯

1-1-3：Serverless的四大核心組件：

我們第一個要說明的服務是「Lambda」，AWS Lambda 是 無伺服器（Serverless）計算服務，允許你執行程式碼而不需要管理伺服器。你只需上傳程式碼，Lambda 會根據觸發條件執行它，並在執行完成後自動釋放資源，僅收費執行時間與請求數量，無需支付閒置成本。

Serverless 的四大核心特性
• 無需配置或管理基礎架構（無需配置、操作、修補等伺服器）
• 按消耗單位自動縮放（按工作/消耗單位縮放，而不是按伺服器單位縮放）
• 按價值計費模式（只需按該單位付費，而不是按伺服器單位付費）
• 內建可用性和容錯能力（無需為可用性進行架構設計）

1-1-4：Lambda 函數範例程式碼：我們用Python來說明這個框架內容

```
import json

def lambda_handler(event, context):
    # Handler() function - Lambda 入口函數
    # event   = 觸發來源傳入的事件數據（dict）
    # context = AWS 執行環境資訊（request ID、記憶體等）

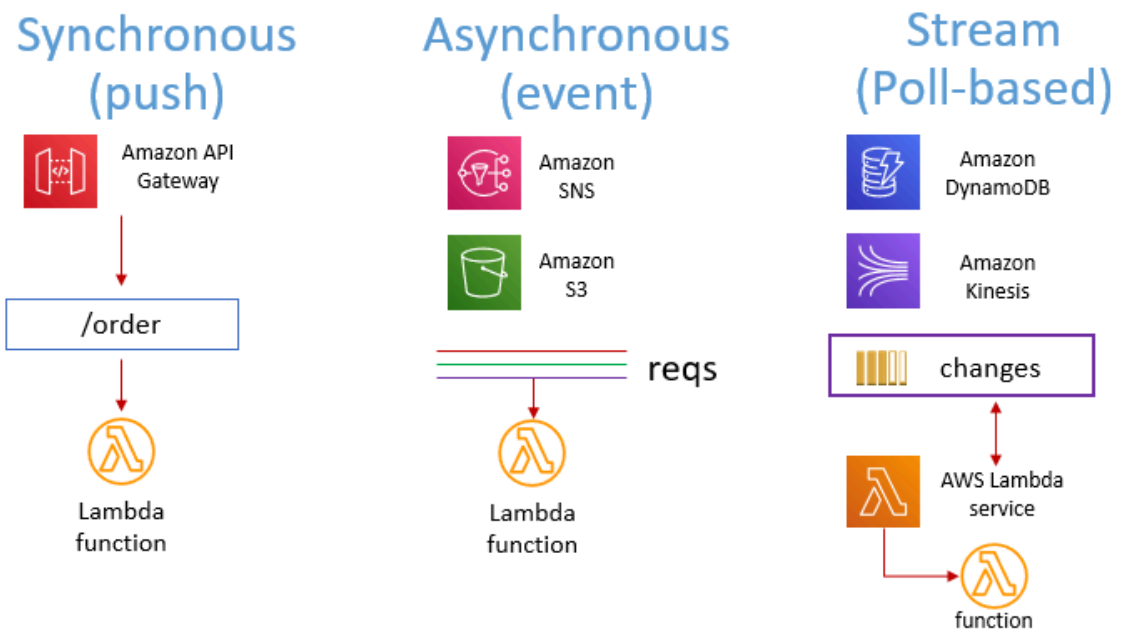
    return {
            'statusCode': 200,
            'body': json.dumps('Hello World!')
    }
```

1-1-5：各部分說明：

import json	載入 Python 內建的 JSON 模組，用來處理 JSON 格式的資料轉換
def lambda_handler(event, context)	這是 Lambda 的**入口函數**，AWS 觸發 Lambda 時會自動呼叫這個函數。函數名稱 `lambda_handler` 是慣例命名，可以改名，但要在 Lambda Console 的 Handler 設定中同步修改 event：觸發來源傳進來的資料，格式是 dict context：AWS 自動注入的執行環境資訊
context.aws_request_id	這次請求的唯一 ID
context.function_name	函數名稱
return	回傳一個 dict 作為函數的執行結果，HTTP 狀態碼，200 代表執行成功
body	回傳的內容，`json.dumps()` 會把 Python 字串轉成 JSON 格式的字串

2-1：Lambda的執行模式

2-1-1：Lambda 支援三種不同的觸發與執行模式，適用於不同場景



執行模型	觸發來源示例	特性說明
Synchronous（同步）	Amazon API Gateway	呼叫者等待 Lambda 回應，適合 API 請求場景
Asynchronous（非同步）	Amazon SNS、Amazon S3	Lambda 以佇列方式異步執行，不等待回應
Stream Poll-based（串流輪詢）	Amazon DynamoDB、Amazon Kinesis	Lambda 定時輪詢事件流，適合大量數據處理

2-1-2：計費方式

Lambda 的費用由兩個部分組成（以奧勒岡 us-west-2 為例）

官方定價參考：*https://aws.amazon.com/tw/lambda/pricing*

📦 請求數量費用	⏱️ 執行時間費用
前 100 萬次請求免費	計算方式：記憶體（GB）× 執行時間（秒）× 單價
超出後每 100 萬次請求：約 $0.20 USD	範例：128MB 記憶體，執行 1 秒，費用約 $0.000000208 USD

2025年後的政策改變

項目	舊政策（2025.07 以前）	新政策（2025.08 以後）
INIT Phase 是否計費	否，不收費	是，根據實際運行時間收費
影響範圍	RunTime + Zip Deploy	RunTime + Zip Deploy + INIT Phase

INIT 階段是 Lambda 冷啟動時執行的初始化流程，包含：

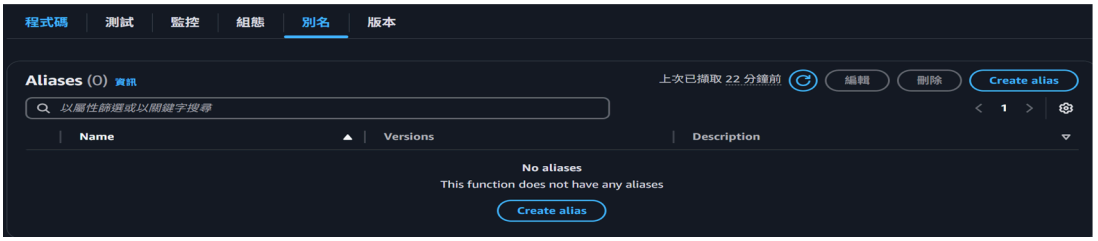
- 載入並解壓縮 ZIP 或容器映像檔，把程式碼或容器部署載入到執行環境
- 執行初始化邏輯（例如 import 模組、連線資料庫、載入大型模型等）
- 建立 Lambda 執行環境（準備 OS、記憶體、網路，尤其 VPC 情況下更耗時）



因應策略:

減少 INIT Phase 影響的建議作法
• 減少冷啟動時間，增加呼叫頻率(保持 Lambda 熱機狀態)
• 改用 Container Image 建立 Lambda(更精確控制啟動環境)
• 改用 Alias 別名(透過 Provisioned Concurrency 預熱)





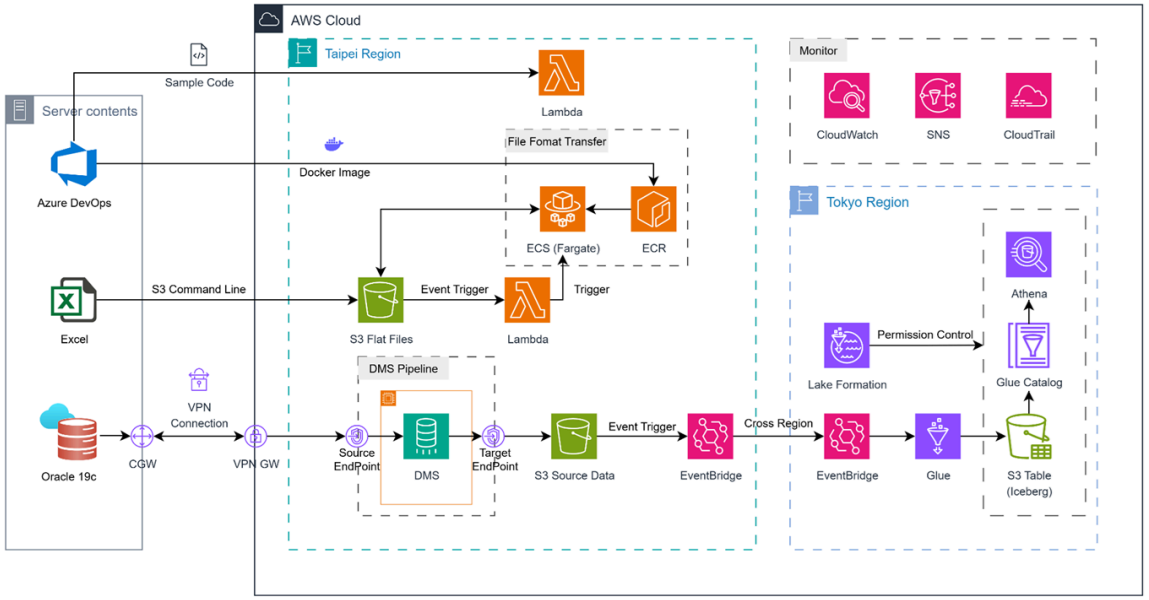
2-1-3:監控方式

AWS Lambda 提供完整的監控工具，幫助開發者追蹤函數執行狀況，同時也可以結合CloudWatch服務來滿足監控的指標，我們可以參考以及設計以下回傳的指標

Requests(請求數)	Lambda 函數被觸發的總次數
Errors(錯誤數)	執行期間發生例外或逾時的次數
Latency(延遲)	函數從觸發到執行完成的時間(毫秒)
Throttles(節流次數)	因達到並發限制而被拒絕的請求次數

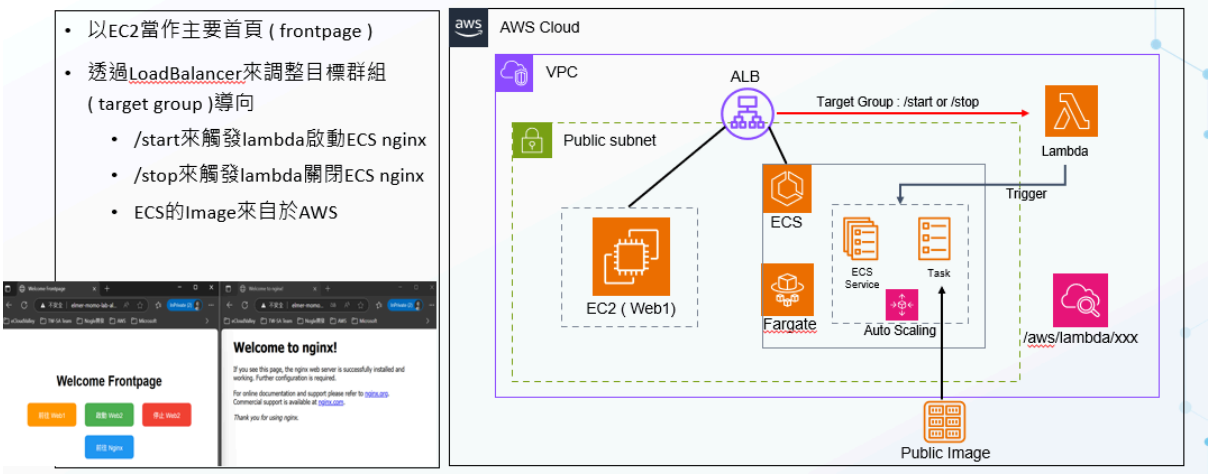
2-1-3：Lambda可以搭配的情境（一）

Lambda 可整合到完整的數據管道中，擔任各階段的觸發與處理角色。

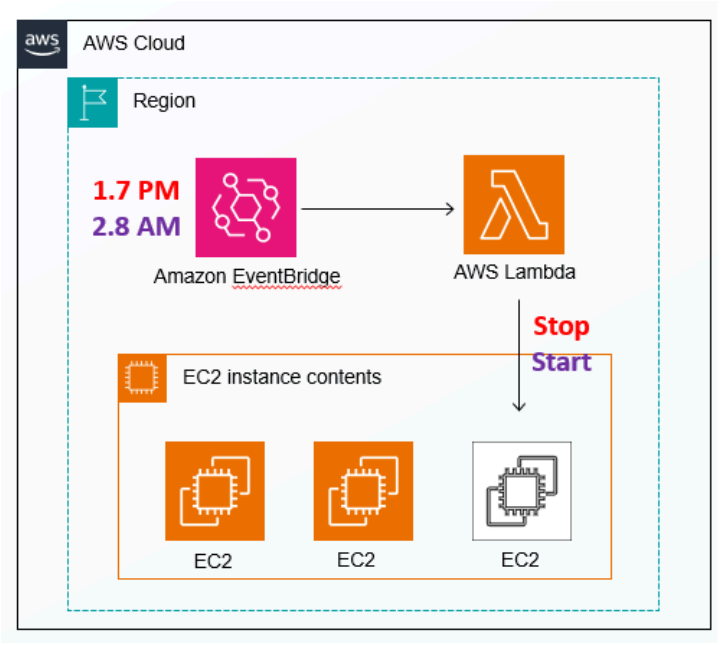


2-1-4：Lambda可以搭配的情境（二）

透過 Application Load Balancer（ALB）觸發 Lambda，進而控制 ECS 服務的啟停，調整由EC2作為網頁服務的一環，在離峰時間也能降低費用產生。



3-1:將EC2主機設定排程關機 09:05開始



透過實作，完成以下自動化排程任務：

- 建立 Lambda Function，停止指定 EC2 實例
- 驗證 Lambda 可正常執行（StatusCode = 200）
- 設定 EventBridge 排程（晚上 7 點自動關機）
- 將 EventBridge 加入 Lambda 的 Trigger

Step 1：建立 Lambda Function（停止 EC2）

方法一：直接指定 EC2 Instance ID

```
import boto3

def lambda_handler(event, context):
    ec2 = boto3.client('ec2', region_name='us-east-1')

    # 指定要停止的 EC2 Instance IDs
    instance_ids = ['i-xxxxxxxxxxxxxxxxx', 'i-yyyyyyyyyyyyyyyyy']

    response = ec2.stop_instances(InstanceIds=instance_ids)

    return {
           'statusCode': 200,
           'body': f'Stopped instances: {instance_ids}'
    }
```

方法二：透過 Tag 標註（進階方式，省去維護 Instance ID）

```
import boto3

def lambda_handler(event, context):
    ec2 = boto3.client('ec2', region_name='us-east-1')

    # 透過 Tag 篩選要停止的 EC2
    response = ec2.describe_instances(
           Filters=[
           {'Name': 'tag:AutoStop', 'Values': ['true']},
           {'Name': 'instance-state-name', 'Values': ['running']}
           ]
    )

    instance_ids = []
    for reservation in response['Reservations']:
           for instance in reservation['Instances']:
             instance_ids.append(instance['InstanceId'])

    if instance_ids:
        ec2.stop_instances(InstanceIds=instance_ids)

    return {'statusCode': 200, 'body': f'Stopped: {instance_ids}'}
```

Step 2：驗證 Lambda 執行結果

透過 AWS CLI 確認 EC2 狀態變化：

```
# 確認目前運行中的主機
aws ec2 describe-instances \
  --filters 'Name=instance-state-name,Values=running' \
  --query 'Reservations[*].Instances[*].[InstanceId,State.Name]' \
  --output table

# 執行 Lambda 後確認主機已停止
aws ec2 describe-instances \
  --filters 'Name=instance-state-name,Values=stopped' \
  --query 'Reservations[*].Instances[*].[InstanceId,State.Name]' \
  --output table
```

Step 3：設定 EventBridge 排程

在 Amazon EventBridge 建立排程規則，目標指向 Lambda Function：

Function Name	LAB-Stop-EC2-Instance
排程規則名稱	stop-ec2-nightly
排程類型	Cron-based schedule
Cron 表達式	cron(0 11 * * ? *) ← UTC 11:00 = 台灣 19:00
目標服務	AWS Lambda Function
觸發設定	在 Lambda Console → Configuration → Triggers 新增

更新：找到建立的LambdaRole 並且加入以下的權限，因為此角色沒有增加EC2的權限所以會失敗

```
        {
        "Effect": "Allow",
        "Action": [
            "ec2:StopInstances",
            "ec2:DescribeInstances"
        ],
        "Resource": "*"
    }
```